

Distributed Systems

Distributed System: a collection of independent and dynamic components which work together and appear to users as a single coherent system.

Benefits:

- Can reuse existing components or services, e.g. Google Maps integration
- Can link up many low-cost machines to work together as a single system
- Can improve reliability and availability by providing redundancy (replication)

Transparency: concealing the system's distributed nature from users and application developers to create the illusion of a single coherent system.

Achieved by creating a single frontend for the collection of components that handles all user interactions with the system.

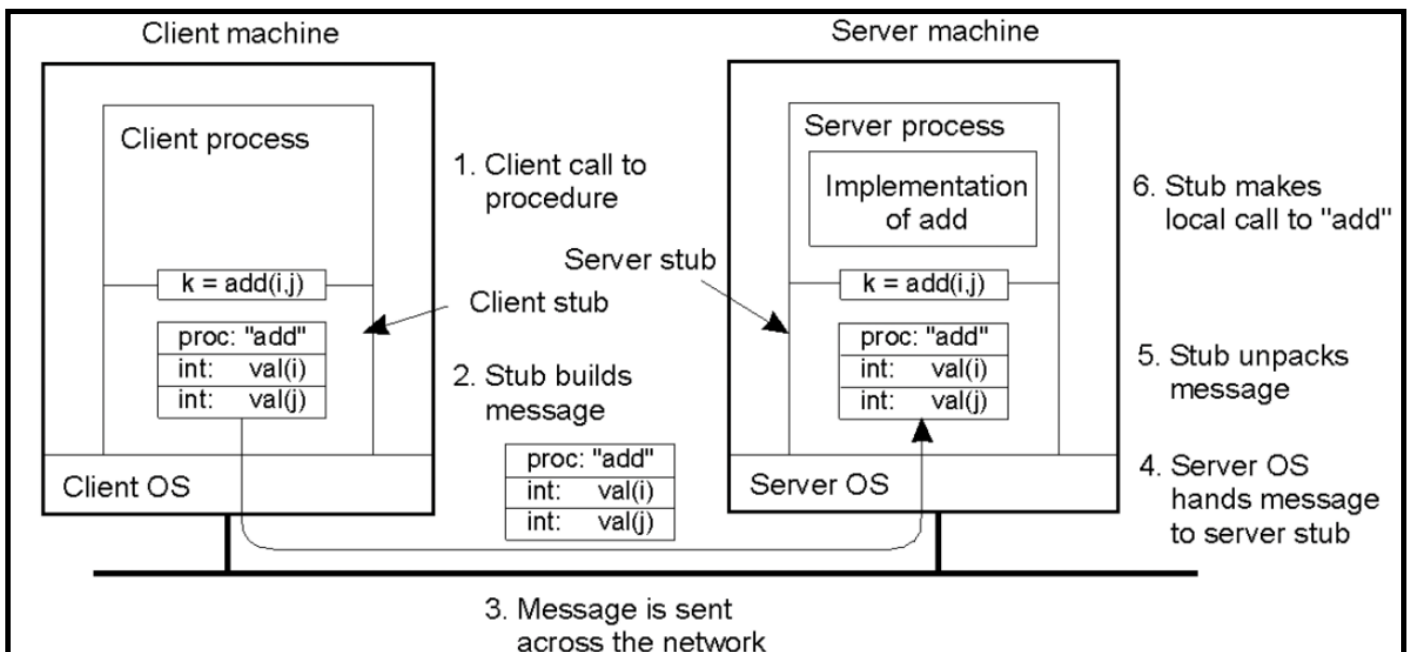
Types of transparency	Description
Access	Hide differences in data representations and how a component is accessed, local and remote resources are accessed through identical operations, e.g., Middleware
Location	Hide where a component is located
Migration	Hide that a component or a resource may move to another location (content distribution network)
Relocation	Hide that a component or a resource may be moved to another location while in use (caching)
Replication	Hide that a component or a resource may have multiple copies (provide fast or local access)
Concurrency	Hide that a resource may be shared by several competitive users (handle shared resources)
Failure	Hide the failure and recovery of a resource or a component

Distributed Systems face challenges, including network issues, data consistency, unpredictable component or communication failure and security. Overcoming these challenges is necessary for a distributed system to work in the real world.

Middleware: Software that allows DS components to work together and communicate as if they were non-distributed. Hides the distributed nature of the system from users and abstracts away low level networking details from developers.

Remote Procedure Call (RPC): masking remote function calls as local ones.

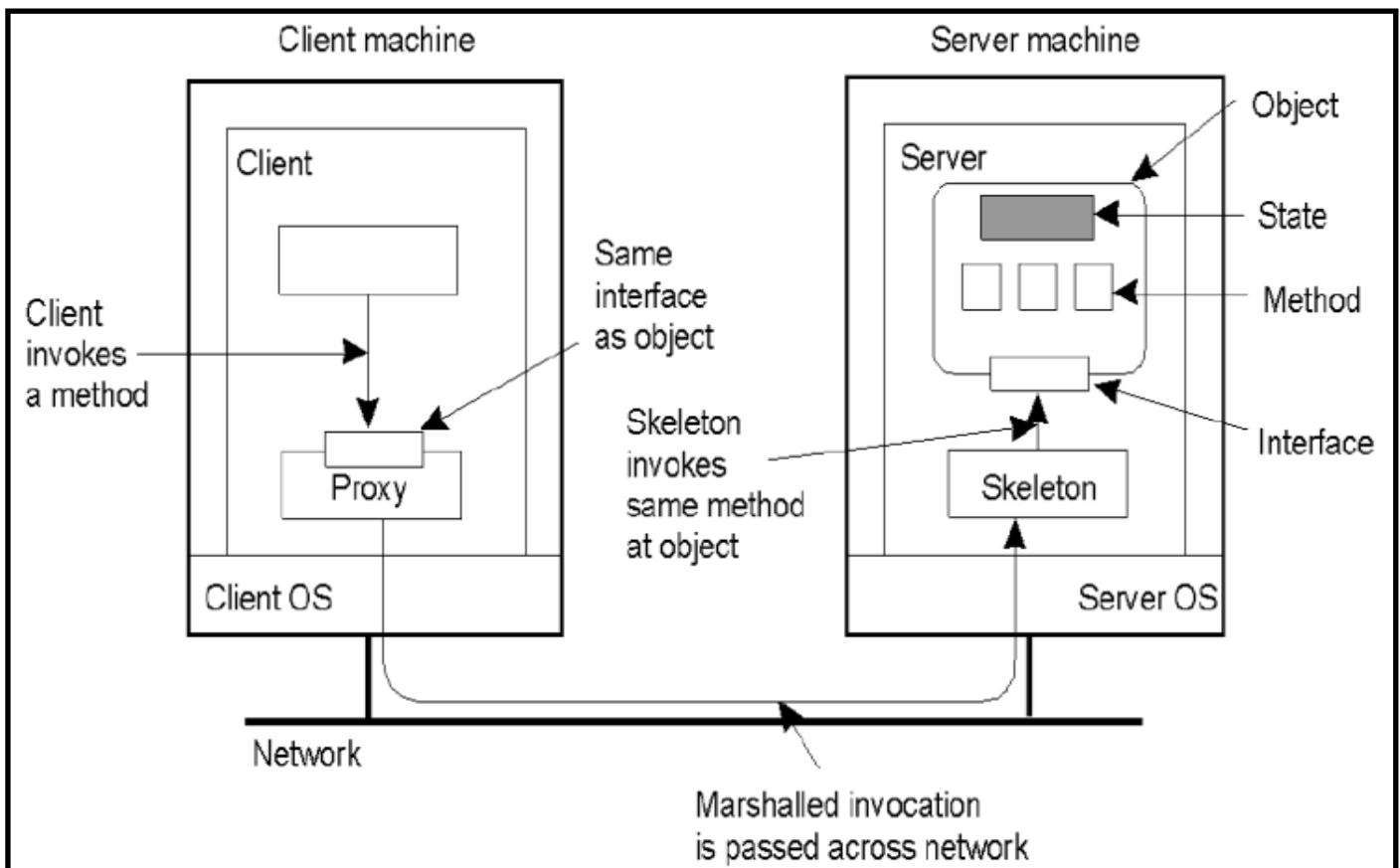
RPC works by using an Interface Definition Language to specify names, parameters, types etc for all server-side functions that can be called remotely. This is then read by the Stub Compiler, which produces a Server-side stub and a client-side stub. The server-side stub is linked to the service functionality, while the client-side stub is linked to the client program that is calling the remote function. All communication details are handled by the stubs, and the RPC library abstracts the details of parameter encoding etc from the programmer.



Limitations:

- Connections are synchronous - held open while waiting for response. Client may be waiting a long time if the server is overloaded and failed clients may delay servers.
- Remote procedure calls are by-value only, so cannot pass arrays, objects or complex data structures.
- Not object-oriented - no support for encapsulation or inheritance, only basic procedural programming (calling functions).

Remote Method Invocation: RPCs applied to OOP Objects, supporting system-wide object references with the actual class containing the implementation on one machine and a shared interface on others. RMI also supports passing object references as parameters.



Proxy is the same as Client Stub, the server stub is known as a Skellington for some reason

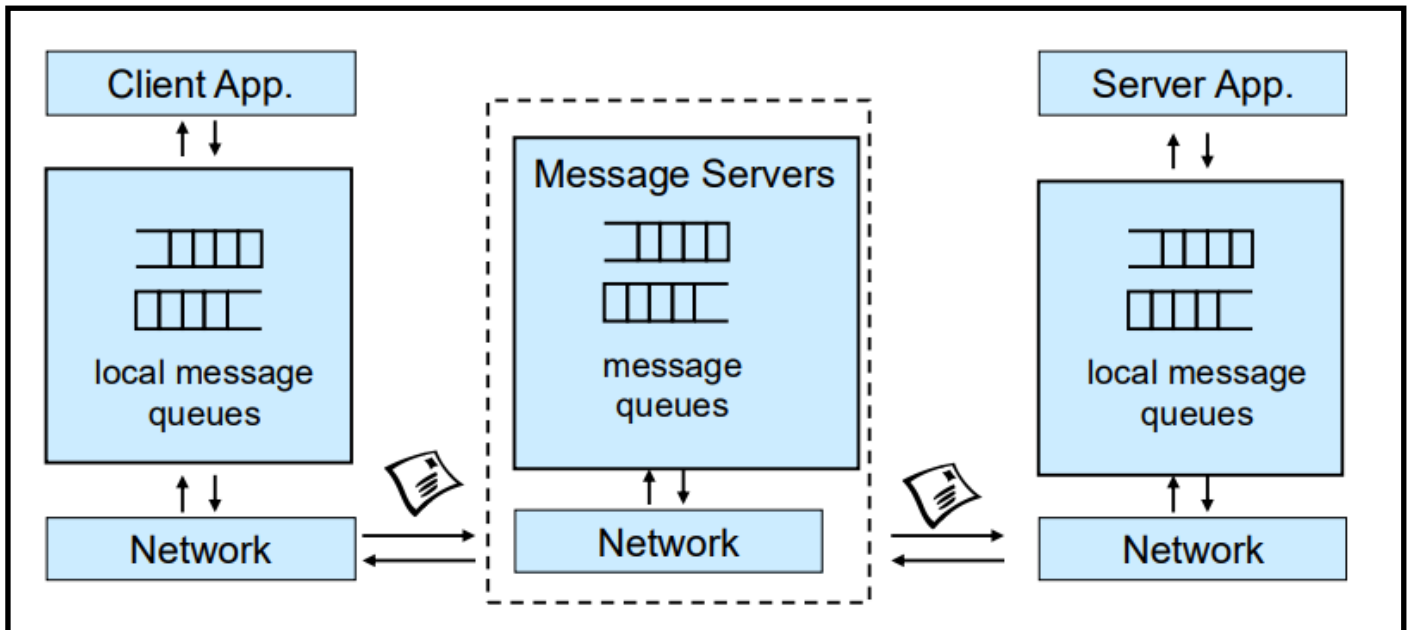
Note: Client-side stubs/proxies *marshal* the data being sent (i.e. serialise it into a bit stream), the server-side stub/skeleton *unmarshals* this bit stream into the relevant data type, passes it to the correct function or method and marshals the results back to the client.

RMI is used in **Object-Oriented Middleware (OOM)** to mask remote objects as being local. In Oom, an Object Request Broker is used to identify and discover remote objects. A Registry or Daemon is some process on some machine in the DS that maintains the names of remote objects. Servers can register their objects; clients can find server objects and obtain remote references and stubs.

OOM provides location transparency since all remote services can be acquired through the request broker, abstracting the geographical complexity of the DS.

Note: communication in OOM is synchronous as with RPC - bad performance.

Message-Oriented Middleware (MOM): more loosely-coupled form of communication using asynchronous messages and intermediate message server/broker (s) to decouple the client and server. Messages are queued.

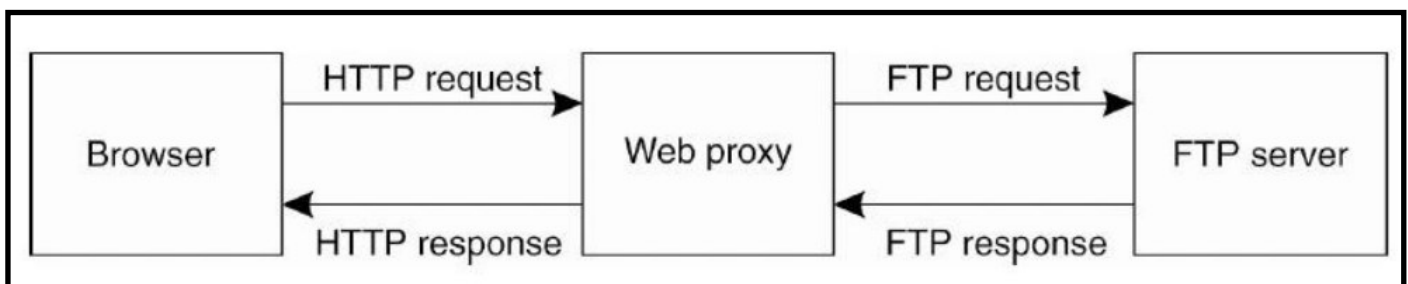


MOM is also reliable, as it retains the message queues even if something dies.

Web Services provide an interface that enables users to communicate with their servers over the Internet. This interface describes the operation of the web service.

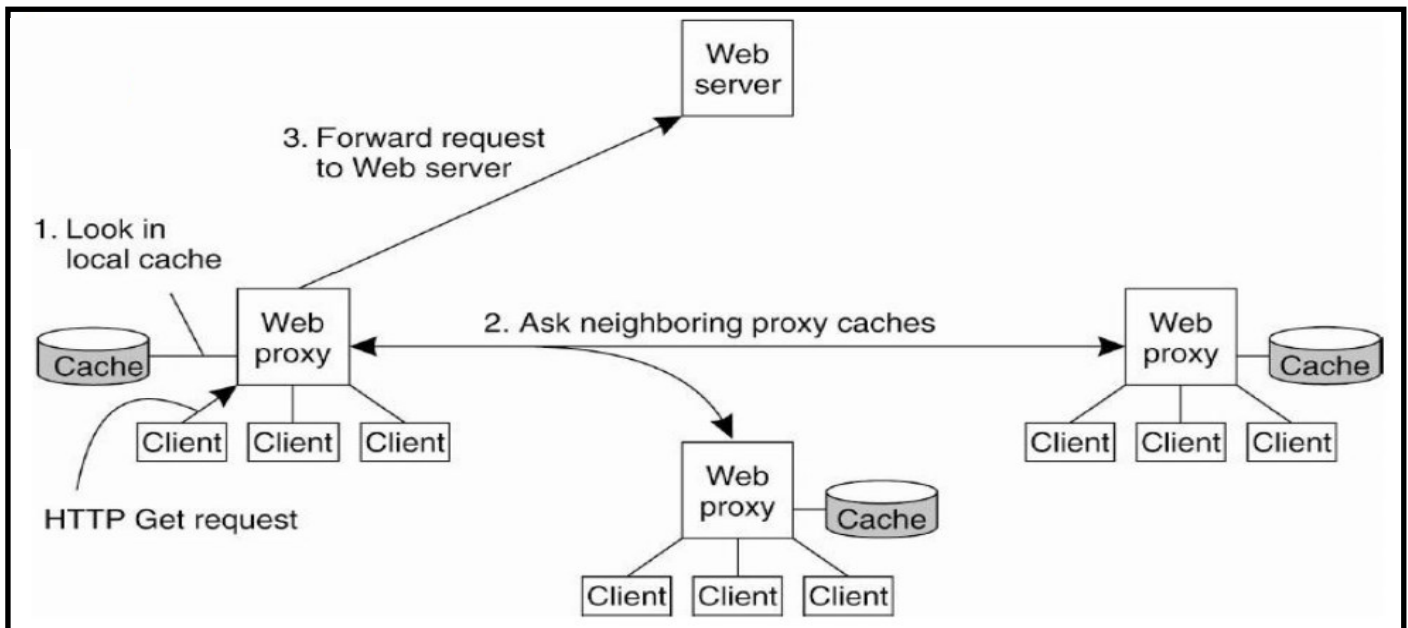
Traditionally, web services used SOAP (Simple Object Access Protocol) and XML to define a common standard that works between any systems, but more modern services use a REST API with JSON. Communication involving web services is performed using HTTP.

Since all communication is done with HTTP, **Proxy Servers** are used to “translate” between HTTP requests and other protocols, e.g. FTP.



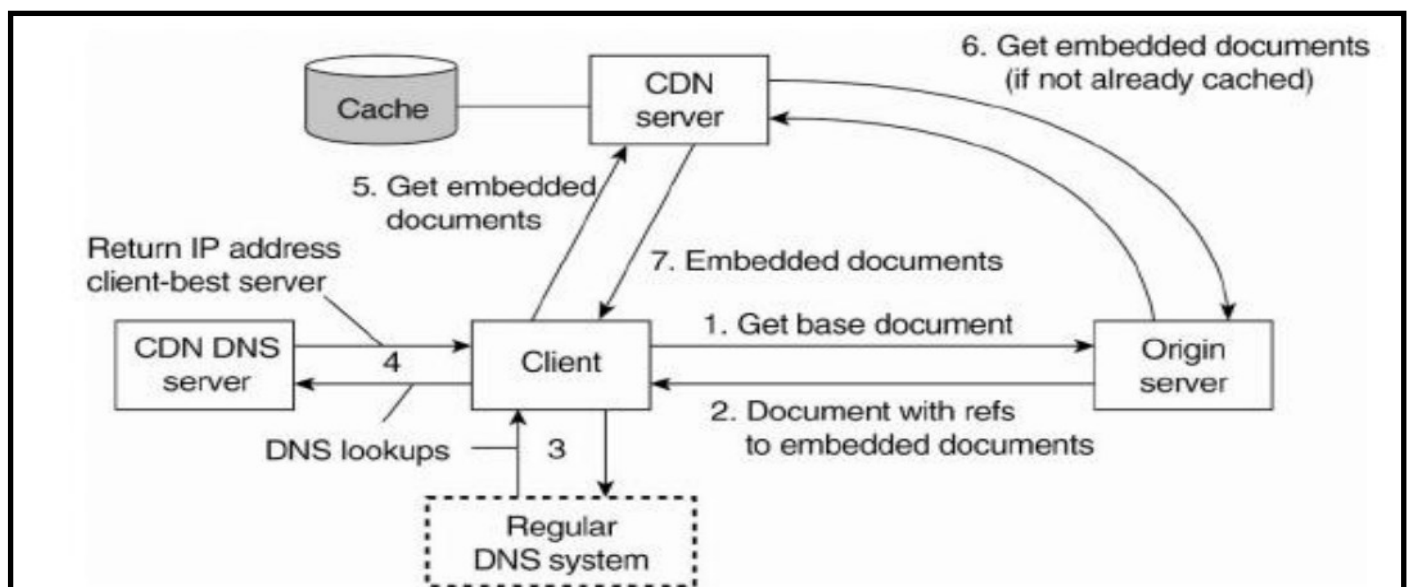
Each proxy has a local cache containing recently/frequently requested resources, allowing for faster fetch times and reducing the load on the actual

FTP server. The principle of **cooperative caching** improves upon this by also checking the cache of other nearby proxy servers before forwarding to the actual server.



Using proxies in this way comes with challenges, for example keeping the caches consistent with the original data, especially if it changes often. Depending on the system, proxies can either periodically request updates from the main server, or the server can push updates onto the proxies.

The process of deploying caches “closer” to the clients in a distributed system is called **Edge Computing**, causing lower latency, higher bandwidth and increased tolerance of network failure. This is employed by **Content Delivery Networks** (CDNs) who maintain a large global network of edge caches, usually for images, videos etc. When a client requests for a resource from the CDN, the closest edge proxy delivers it.



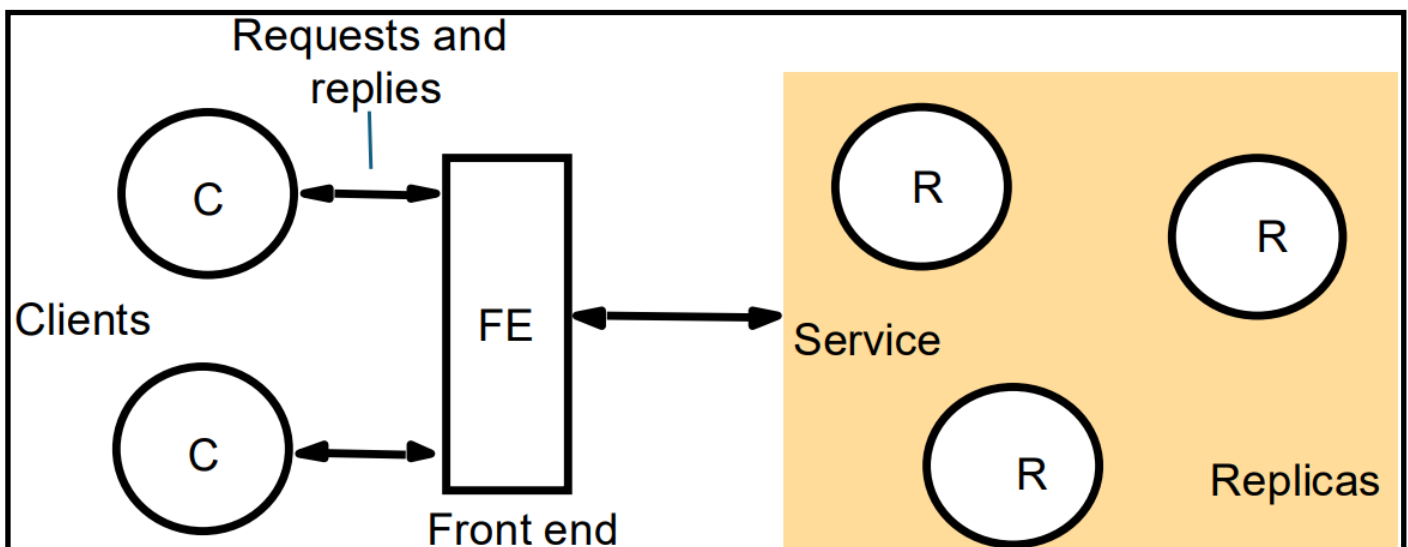
Replication and Fault Tolerance

Replication is where copies of data or functions are stored in multiple components of a distributed system. This is often done to increase reliability (fault-tolerance) and improve performance (through load-sharing).

There are two main approaches to handling replication: either have the application be responsible for replication, updating all copies and maintaining consistency, or have the middleware handle it to simplify application development.

Replication can involve Computation Replication - executing the same functional process (with potentially different implementations, language, hardware etc) across multiple instances or storing the same pieces of data on multiple devices.

Note: replicas may not necessarily be consistent all of the time (e.g. updates take time to propagate).



Clients send requests to the frontend, which monitors replica availability and distributes client requests between the replicas. Read requests are handled by only one replica, while write requests usually require synchronisation between all replicas to maintain concurrency control. One or more replicas send a response to the frontend, which processes the response before returning it to the client.

Faults: Inconsistent results can be obtained from distributed servers as a result of:

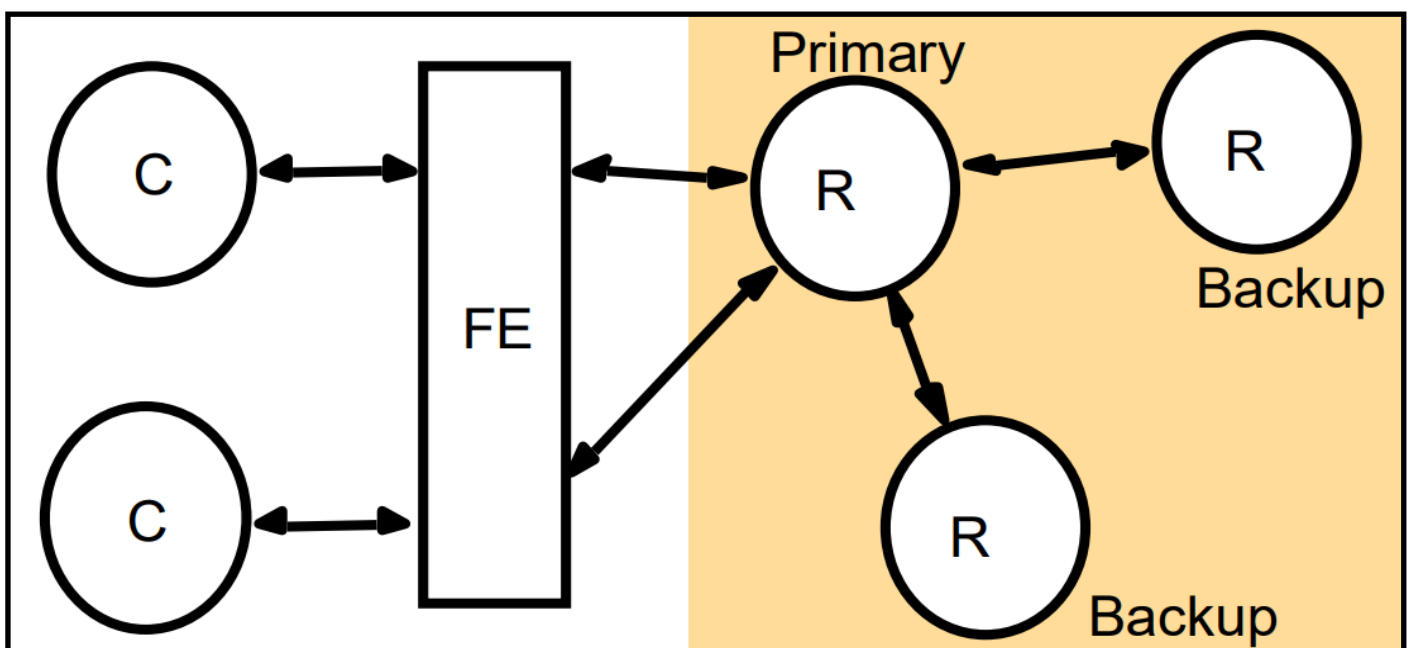
- **Omission Failures** - failing to receive a request or send a response, e.g. because of a crash
- **Commission Failures** - processing a request incorrectly, sending incorrect responses or corruption of local state, either due to malfunctions, bugs or malware.

A **Fault Tolerant** service is one which provides a correct service despite the occurrence of server failures. Each replica is assumed to either behave as expected or to have crashed (i.e. no software bugs). A service is said to be correct if it has Failure Transparency (it keeps responding even if failures have occurred) and Location and Relocation Transparency (clients cannot tell the difference between the service provided by different replicas).

Replication strategies for Fault Tolerance:

Passive Replication (primary-backup model):

At any time there is a single primary replica and one or more secondary (backup) replicas. The frontend communicates with the primary replica, which executes the required operations and propagates the updated data to the backups. The primary waits for acknowledgement from the backups before sending its response to the frontend. If the primary fails, one of the backups becomes the new primary.



Passive Replication with n replicas can provide a correct service with up to $n-1$ replica crashes. It requires little additional frontend functionality - only need to implement a lookup for a new primary when the current one dies. However, the system overhead of the data propagation is large.

Summary:

- Strong consistency
- Fault-tolerance provided by backups
- Backups not utilised unless primary fails, wasting resources
- Simple to implement

Active Replication:

All replicas play the same role. They all start in the same state. The frontend multicasts each request to *all* replicas, they all perform the same operations in the same order and each sends its response back to the frontend. The

Byzantine Failure - an arbitrary error that occurs during program execution on a Distributed System where component(s) return the incorrect result, e.g. due to software bugs or hackers.

frontend may return the first response it receives or wait for more - if we just want crash-resilience, return the first response as soon as it is received; if we want Byzantine failure tolerance we need to wait for a majority response.

Active replication requires a reliable multicast to be available such that all replicas receive the requests in the correct order. It is tolerant to $(n/2) - 1$ Byzantine failures.

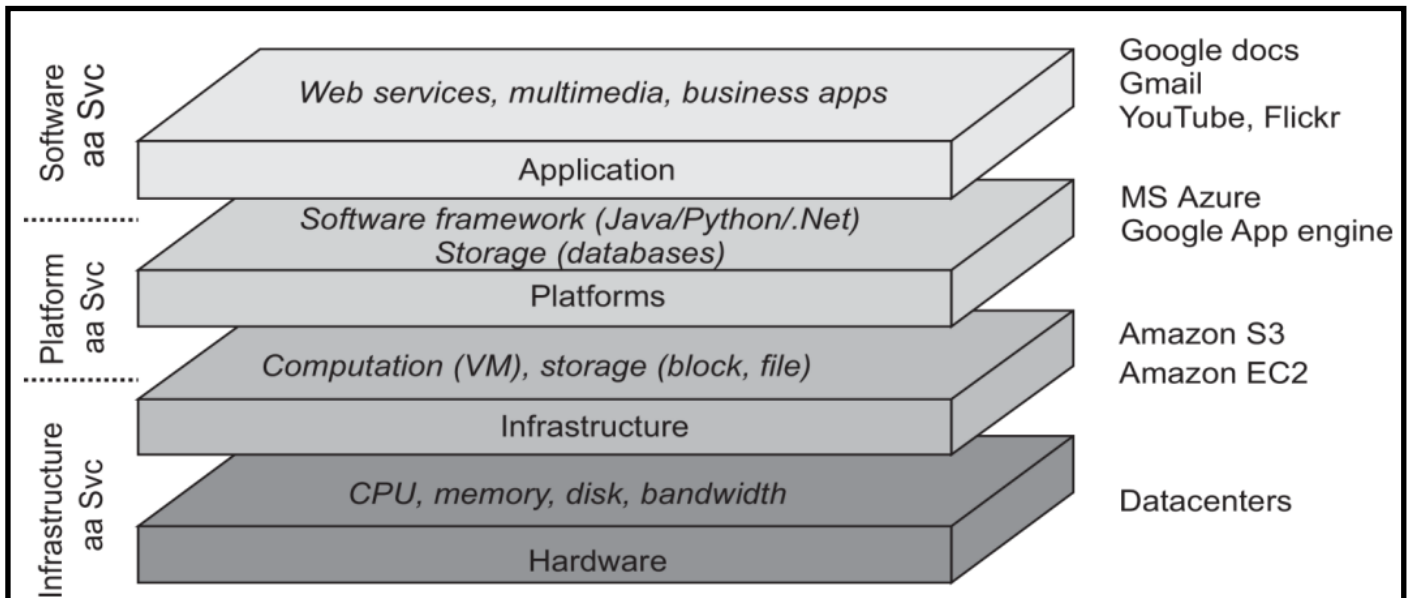
Summary:

- Complex to implement - requires reliable ordered multicast
- If multicast unreliable, requests could arrive out of order leading to inconsistency
- Used when high availability of servers is needed, is resilient to Byzantine faults as all servers process all requests

Cloud Computing and Load Balancing

A **Cloud** is one of those white fluffy things in the sky - if you go outside you would know.

Cloud Computing is the use of remote servers to run distributed applications.



There are four layers:

- Hardware: processors, memory, power etc.
- Infrastructure: virtualisation, i.e. virtual machines, virtual storage that can be rented out for use, e.g. datacentres.
- Platform: higher level abstractions/APIs for storage and resources, e.g. operating system and programming environment for developers to deploy their apps on
- Application: fully developed software such as office suites hosted on a cloud platform, for use by ordinary idiots.

Traditionally, datacentres used a static architecture, with applications running on physical servers managed manually by system administrators, whereas modern datacentres run applications inside virtual machines, with flexible dynamic mapping between physical and virtual resources and increased automation.

Virtualisation involves creating virtual representations of physical hardware,

mimicking its functionality and enabling multiple virtual machines to be run simultaneously, each with its own OS and applications. This allows for easier portability, and isolation of different components - if one VM becomes compromised the rest of the datacentre isn't affected. We can also provide virtual interfaces for specific hardware, e.g. for older software to be compatible with new hardware as the hardware is updated. CPU instruction set architectures, system and library calls and operating systems can all be mimicked.

Load Distribution

Load Sharing: reducing the likelihood of there being idle servers.

Load Balancing: equalising the load at all servers.

Load Distribution algorithms aim to minimise latency and energy consumption, and maximise the chance of requests being accepted. These can take the form of distributing or redistribution tasks among servers and/or organising servers to work collaboratively for load sharing.

Task transfer can be preemptive (transferring tasks that have not yet started) or non-preemptive (transferring partially executed tasks). Preemptive is good for load sharing but not load balancing, non-preemptive is better for load balancing but is more expensive as it involves transferring the entire process states and virtual memory contents.

Load Distribution can either be **static** (fixed, not dependent on the state of the system), e.g. round robin, **dynamic** (fixed policy but scheduling decisions depend on the system state), e.g. Join-Shortest-Queue / Least-Workload policy, or **adaptive** where policies change dynamically

Load Distribution Algorithms have four main components:

- **Transfer Policy:** decides when a server needs to transfer tasks (thresholds)
 - a server becomes a sender when its load exceeds some threshold and becomes a receiver when its load is under some other threshold
- **Selection Policy:** determines which tasks to transfer, e.g. based on

estimated task execution time or to minimise location-dependent system calls.

- **Location Policy:** decides which receiving server to transfer a task to, usually by using polling to find a lightly loaded server.
- **Information Policy:** decide when, where and what information to collect about other servers, e.g. demand-driven, periodic, state-change driven.

Example Algorithms:

Sender-Initiated:

- Transfer Policy - a server is a *sender* if the queue length exceeds a threshold T , and a *receiver* if accepting a task would not cause its queue length to exceed T .
- Selection Policy - Only newly arrived tasks - non-preemptive.
- Location Policy - Tasks transferred to a server at random, no need for state collection but unnecessary transfers may occur.

Alternatives: Poll servers until a receiver is found, the receiver must accept the task.

Poll servers, select the receiver with the shortest queue

(set a poll limit to reduce overhead in both cases)

- Information Policy - Demand-driven: a server only collects the state of other servers when it becomes a sender or receiver.

Issue: At high loads, where there are more senders than receivers it will become difficult for senders to find receivers, resulting in significant polling that could make the system unstable.

Receiver-Initiated:

- Transfer Policy - a server is a *sender* if the queue length exceeds a threshold T , and a *receiver* if it is under T .
- Selection Policy - Newly arrived tasks with a high priority, and preemptive task transfer with a lower priority

- Location Policy - Receivers poll random servers until a sender is found, then transfer from the sender to receiver.

Alternatives: Poll a number of servers and choose the highest-loaded sender.

Repeatedly attempt to transfer tasks from another server until a poll limit is reached.

- Information Policy - demand-driven (as sender-oriented)

Issue: Since polling is initiated by the receiver, senders may have started executing new processes before transfer occurs, resulting in more preemptive transfers which are more expensive.

Symmetric Algorithm:

Senders search for receivers and vice versa, so that at high loads senders can easily find receivers (as receivers will actively find senders), and at low loads receivers can easily find senders (as senders will actively find receivers).

However, this may also result in the worst of both worlds, with excessive polling at high loads impacting stability and receiver-initiated transfers often being preemptive.

Adaptive Algorithm:

Each server actively keeps track of the status of all servers, maintaining three lists of *Senders*, *Receivers* and *OK* (neither senders or receivers). In the initial state, all servers are assumed to be Receivers.

- Location Policy at Sender: Poll the head of the *Receivers* list. The polled server puts the sender at the head of its *Senders* list and informs the sender of its current status (sender, receiver, OK). If the polled server is still a receiver, transfer the task to it, otherwise update the lists accordingly and poll the new head of the *Receivers* list. If the *Receiver* list is empty, stop - transfers can still occur in a receiver-initiated fashion later.
- Location Policy at Receiver: Poll from head to tail of the *Senders* list. If no sender is found, poll from *tail to head* in the *OK* then *Receivers* list. Transfer a task if a sender is found, otherwise update lists. Polling stops when a

sender is found, or some poll limit is reached.

Note: we go *tail to head* in the *OK* and *Receivers* lists since the most up to date information is in the head. We are hoping that the least up to date ones have changed status to Sender since it was last updated.

At high loads, sender-initiated polling reduces as the receiver list gets smaller and receiver-initiated polling takes over and can find a sender easily. At low loads, sender-initiated polling takes over and senders can easily find a receiver. Receivers being unable to find senders at low loads has little impact on performance since the system is under low load.

In general, sender-initiated algorithms work better if a system never gets highly loaded, or if there is a high cost for preemptive transfers; receiver-initiated algorithms are preferred for systems that always have a high load, symmetric algorithms are better for widely fluctuating loads with low cost of preemptive transfer, and adaptive algorithms are best for heterogeneous work (where the type and resource usage of tasks varies significantly).

When the number of servers becomes large (e.g 10000), these algorithms become unusable. We have to use alternative algorithms:

- Power-of-d: Select d servers, chosen uniformly at random, and assign the new task to the server of these d with the smallest queue.
- Join-Idle-Queue: With a large number of servers, there is a high chance one is idle. Servers notify the Load Balancer when they become idle, the load balancer then assigns an incoming task to that server. If no idle server, just pick a random one.

Concurrency Control

Shared objects may be simultaneously accessed by multiple processes or events. Servers managing shared objects are responsible for maintaining consistency. This is often done through **Locking**:

Locks are held in a server locally, the decision to grant a lock is made by the local Lock Manager. However, with distributed transactions, locks need to be acquired for resources on different servers, and these cannot be released until the transaction either commits or aborts at all servers involved in the transaction. This can lead to deadlocks.

We could also use **Timestamping**, where each transaction is assigned a timestamp when it starts, and transactions are serialised in timestamp order on all servers to maintain consistency. However, this requires a synchronised global clock to ensure the order is the same across all servers. Network Time Protocol is used to synchronise the clocks of computers across a network.

Replication leads to additional challenges for consistency, as network latency and transmission errors could lead to operations being received in the wrong order at some replicas.

A **Consistency Model** defines rules for the apparent order and visibility of updates. For a read operation, all consistency models attempt to return the results of the most recent write.

Strict Consistency:

Any read of a data item X returns a value corresponding to the most recent write on X. Writes are immediately visible to all processes - assume no transmission delays. In practice impossible to achieve on distributed systems or systems with replication due to clock synchronisation issues and network delays.

Linearisability:

There exists some interleaving of operations such that the interleaved sequence produces a single correct copy of the objects and that the order of

operations is consistent with the real times at which the operations occurred in the execution.

Sequential Consistency:

If for any transaction, there is some interleaving of operations such that the interleaved sequence produces a single correct copy of the objects and the order of operations from each process in the interleaved sequence is the same as the order of those operations within the individual processes (but not necessarily between processes).

Suppose that both accounts have balance '0' initially		This is <u>not</u> linearisable since <code>getBalance_A(x)</code> occurs after <code>setBalance_B(x,1)</code> in the execution, so should return 1 not 0. It is, however, sequentially consistent, with order <code>getBalance_a(y)</code> , <code>getBalance_A(x)</code> , <code>setBalance_B(x,1)</code> , <code>setBalance_A(y,2)</code> .
Client 1	Client 2	
<code>setBalance_B(x,1)</code>		
	<code>getBalance_A(y) → 0</code>	
	<code>getBalance_A(x) → 0</code>	
<code>setBalance_A(y,2)</code>		

Unlike Linearisability, Sequential Consistency does not care about the global order of operations, only that the operations from each client/process occur in the correct order and that all clients/processes see the same order.

Causal Consistency:

A weakening of Sequential Consistency, where writes that are potentially causally related must be seen by all processes in the same order, but concurrent writes may be seen in a different order on different machines.

P1: <code>W(x) a</code>	<- This is Causally Consistent since neither of the writes are "causally related", so processes can see them in different orders.													
P2: <code>W(x) b</code>														
P3: <code>R(x) a</code> <code>R(x) b</code>														
P4: <code>R(x) b</code> <code>R(x) a</code>														
This is <u>not</u> Causally Consistent since P2 reads <i>a</i> from <i>x</i> <u>before</u> writing <i>b</i> to <i>x</i> . This means that all processes must see the write of <i>a</i> to <i>x</i> before <i>b</i> . Since P4 sees <i>b</i> then <i>a</i> , this is not causally consistent. ->		<table> <tr> <td>P1: <code>W(x) a</code></td><td colspan="2"></td></tr> <tr> <td>P2: <code>R(x) a</code> <code>W(x) b</code></td><td colspan="2"></td></tr> <tr> <td>P3: <code>R(x) a</code> <code>R(x) b</code></td><td colspan="2"></td></tr> <tr> <td>P4: <code>R(x) b</code> <code>R(x) a</code></td><td colspan="2"></td></tr> </table>	P1: <code>W(x) a</code>			P2: <code>R(x) a</code> <code>W(x) b</code>			P3: <code>R(x) a</code> <code>R(x) b</code>			P4: <code>R(x) b</code> <code>R(x) a</code>		
P1: <code>W(x) a</code>														
P2: <code>R(x) a</code> <code>W(x) b</code>														
P3: <code>R(x) a</code> <code>R(x) b</code>														
P4: <code>R(x) b</code> <code>R(x) a</code>														

Eventual Consistency:

Eventually, all replicas will converge to identical copies. When one or a few

processes perform writes and updates do not need to be made available immediately, eventual consistency is often used, e.g. by cloud storage providers.

Redundancy and Error Recovery

A distributed system has many independent components. In general, if every component has probability of p , then a system with n components has a probability of $1-(1-p)^n$ of having at least one failed component.

System quality is measured by Reliability and Availability:

Reliability is the time duration between failures, measured by *mean time between failures*, MTBF.

Availability is the percentage of time the system is ready to use, measured by $MTBF / (MTBF + \textit{mean time to repair (MTTR)})$

Note: In practice, the frequency and duration of outages are often more important than the *average* availability.

Failures:

A Failure is where a component does not live up to its specification, e.g. a crashed program. An Error is the part of a component that caused the failure, e.g. a software bug. A Fault is the cause of an error, e.g. ChatGPT shitting out slop code.

Failure Types:

Type	Description of server's behavior
Crash failure	Halts, but is working correctly until it halts
Omission failure <i>Receive omission</i> <i>Send omission</i>	Fails to respond to incoming requests Fails to receive incoming messages Fails to send messages
Timing failure	Response lies outside a specified time interval
Response failure <i>Value failure</i> <i>State-transition failure</i>	Response is incorrect The value of the response is wrong Deviates from the correct flow of control
Arbitrary failure	May produce arbitrary responses at arbitrary times

Redundancy - applying duplication to increase reliability. This can either be done using Active/Passive Replication (as discussed earlier) or through a more operational approach.

Time Redundancy:

Designing a system such that an action can be performed again if something went wrong the first time. Typically used when faults are transient or intermittent (i.e. happen randomly and rarely). Achieved by performing the same operation multiple times and comparing results - this can impact system performance and only detects temporary faults.

Component Redundancy:

Introducing two or more independent components that perform the same functionality. Achieved using N-Version Programming - providing different implementations of the same program and running them in parallel. This can deal with hardware faults, and software faults in one specific implementation, but not correlated faults.

Information Redundancy:

Adding extra data, i.e. checksums, parity bits, to detect when bits become garbled. This requires no extra hardware, but adds design complexity and has limited capability for correcting faults.

Error Recovery

Error Recovery involves moving the system from an erroneous state to a state that does not contain the error. This happens after the failure has been isolated and the error has been contained such that it doesn't propagate to other parts of the system.

Types of Error Recovery:

Backward Recovery:

Moving the system back to a failure-free state; ensuring the system is correct before continuing operation. This is typically implemented using checkpointing

- where each component/process in the system periodically saves its state such that it can restart execution from that point. A global checkpoint can be set up with a local checkpoint from every process if together they form a consistent global state. The most recent global checkpoint is called the recovery line.

Types of checkpointing:

- Uncoordinated Checkpointing: each process takes checkpoints independently and automatically. Simple but could lead to useless checkpoints and a greater risk of cascading rollback.
- Coordinated Checkpointing: processes take coordinated checkpoints such that they always form a consistent global checkpoint. Less storage overhead since no checkpoints are useless, avoids cascading rollback but there is an impact on performance as all processes have to stop together to perform checkpointing.
- Communication-Induced Checkpointing: processes are forced to take checkpoints based on information contained in application messages received from other processes. Avoids cascading rollback.
- Message Logging: Take infrequent checkpoints but log all messages between checkpoints to a stable local storage. Upon recovery, go to previous checkpoint and replay all logged messages that have happened since then.

Issue: sometimes the recovery line may roll back all the way to the initial state.

Forward Recovery:

Finding a new state from which the operation of the system can continue, aiming to avoid a system from having to stop if a failure occurs. Forward recovery requires knowledge about the error (unlike backward recovery) to take appropriate action, but is more efficient in time and space as no previous state needs to be stored and no operations have to be repeated.